

Problem statement:

Uber drivers often do not have enough clear information about difficult pickup conditions before accepting a ride, which can lead to wasted time, frustration, and a worse driving experience.

Solution:

Create **Pickup Insights**, a feature that shows short, location-based warnings before a driver accepts a ride.

Examples:

- Busy intersection
- No stopping zone
- Apartment entrance hard to find
- Heavy event traffic
- Pickup spot usually crowded

How it helps:

This helps drivers make better decisions, avoid frustrating pickups, and feel more in control while driving, which can improve the overall driver experience and support retention.

PRD

Problem Statement

Uber drivers accept rides without adequate information about pickup conditions at the destination. Difficult pickups, such as confusing apartment complexes, no-stopping zones, construction areas, and event traffic lead to wasted time circling the block, increased frustration, and, in some cases, ride cancellations.

According to Uber's public earnings disclosures, driver churn remains one of the company's highest operational costs, with estimates suggesting it costs Uber thousands of dollars to onboard and activate a single new driver. Even a small reduction in frustration-driven churn would translate into significant savings.

Qualitatively, driver forums and communities (Reddit's r/uberdrivers, driver feedback surveys) consistently surface pickup difficulty as a top complaint, particularly in dense urban areas, college campuses, and event venues. Currently, drivers [see only a pin on a map](#) and an estimated fare before accepting. They have no visibility into what the pickup experience will actually be like.

Impact:

If Pickup Insights reduces driver-initiated cancellations at flagged locations by even 10-15%, this would improve ride completion rates, reduce rider wait times caused by cancellations, and improve driver satisfaction scores. Over time, improved driver experience at pain-point locations supports retention, which is Uber's most cost-effective growth lever on the supply side. Additionally, the data collected through driver reports creates a feedback loop that can improve Uber's mapping and pickup pin accuracy platform-wide.

Proposed Solution

The solution is for Uber to release a new feature called Pickup Insights.

Pickup Insights is a pre-acceptance information layer that gives drivers more context about the pickup experience before they choose to accept a ride. Alongside the pickup pin and estimated fare, drivers would see signals such as difficult parking areas, no-stopping zones, apartment or campus entrances, construction, event traffic, and the best side of the street to stop on. These insights would be generated from historical trip data, driver reports, rider instructions, and real-time conditions. By helping drivers better understand pickup complexity in advance, the feature can reduce cancellations, save time, and improve overall driver satisfaction.

User Persona

Name: Marcus J.

Age: 34

Gender: Male

Tagline: "I just want to know what I'm getting into before I drive there."

Background: Marcus drives for Uber full-time in Philadelphia. He's been driving for about 2 years and completes 25-35 rides per day. He drives during both peak and off-peak hours to maximize earnings. He uses his personal smartphone (Android) mounted on his dashboard while driving.

Experience with product: High. He uses the Uber Driver app daily and is very familiar with the accept/decline flow, surge pricing, and navigation. He doesn't need onboarding; he needs better information.

Goals: Maximize earnings per hour by minimizing wasted time. Avoid frustrating pickups that eat into his time without extra pay. Feel more in control of which rides he takes.

Concerns: Arriving at a pickup and finding no legal place to stop. Spending 5+ minutes trying to find a building entrance. Getting stuck in event traffic he didn't anticipate. Accepting a ride that looks short but turns into a headache.

Quote: "I've been driving long enough to know which spots are a nightmare. I just wish someone told me that before I accepted."

Functional Requirements (what the solution must do)

1. Pickup Insights must appear alongside the location pin and estimated fare before the driver accepts the ride.
2. Pickup insights must display warnings for known pickup challenges as mentioned in the problem statement.
3. Pickup Insights must recommend the best pickup side of the street, entrance or stopping area when available.
4. Driver rider chats must be analysed for any signs of driver frustration or insights on pickup location shared by the rider.
5. The system must use historical trip data, cancellation data, driver reports, rider instructions, and map information to generate pickup insights.
6. When a driver completes a ride, he/she must be prompted to fill in 3 questions
 - a. On a scale of 1–10, how easy was it to find the rider?
 - i. slider
 - b. Was the pickup location accurate?
 - i. Yes / No
 - c. Were there any pickup issues encountered?
 - i. Wrong pickup location
 - ii. Heavy traffic
 - iii. Construction area
 - iv. No Stopping/Waiting Zones
7. Drivers must be able to submit optional written comments or photos about the pickup location.
8. Pickup Insights must continuously update based on newly submitted driver feedback and changing road conditions.
9. Riders must be able to receive prompts encouraging them to provide clearer pickup instructions at locations with repeated pickup issues.
10. The system must flag pickup locations with high cancellation rates or repeated driver complaints for internal review.

Non Functional Requirements (how the solution must behave)

1. Pickup Insights must load within 1–2 seconds after a ride request is received.
2. The feature must not noticeably slow down the ride acceptance flow.
3. The system must maintain high accuracy so that outdated or incorrect pickup insights are minimized.
4. Pickup Insights must be easy to understand at a glance while driving, using short phrases, icons, and color-coded warnings.
5. The system must protect driver and rider privacy by anonymizing any analyzed chat messages or feedback data.
6. The system must be scalable to support millions of ride requests and pickup locations globally.
7. Pickup Insights must be designed to minimize distraction and should not require drivers to read long paragraphs while driving.
8. The feature should continue functioning even if only limited historical data is available for a pickup location.

User Stories

Viewing Pickup Insights

As a driver, I want to see a short warning about difficult pickup conditions on the ride request card so that I can make a more informed decision before accepting.

As a driver, I want to tap on the insight badge to see a detailed breakdown so that I can understand the severity and type of issue at that location.

As a driver, I want insights to only show when they're recent and relevant so that I'm not seeing outdated warnings about problems that no longer exist.

As a driver, I want the insight badge to not interfere with my accept/decline timer so that I don't accidentally miss a ride while reading the details.

Reporting Pickup Conditions

As a driver, I want to submit a quick pickup report after completing a ride so that I can flag difficult conditions without it eating into my driving time.

As a driver, I want to choose from predefined tags like "No stopping zone" or "Entrance hard to find" so that I can report issues quickly without typing anything.

As a driver, I want my report to be saved locally if I have poor connectivity so that I don't lose my feedback when driving through low-signal areas.

Incentive Balancing

As a driver, I want to see an adjusted fare boost for pickups flagged as difficult so that I feel compensated for accepting a ride I know will be more effort.

As a rider, I want to receive a suggested alternative pickup pin when my location has been flagged so that my driver can find me more easily and I don't wait longer than necessary.

Testing Plan & Acceptance Criteria

Test Case 1 — Pickup Insight Display (Happy Path)

- Action: Driver receives a ride request at a location that has 5+ driver reports flagging "Building entrance hard to find."
- Expected: A short insight badge appears on the ride request card (e.g., "Drivers report: Entrance hard to find") before the driver taps Accept or Decline.
- Acceptance Criteria: The insight badge renders within 1 second of the ride request appearing. Badge text matches the highest-frequency report tag for that location.

Test Case 2 — No Insights Available (Happy Path)

- Action: Driver receives a ride request at a location with no driver-submitted reports and no historical GPS anomalies.
- Expected: The ride request card displays as normal with no insight badge. No empty/blank badge placeholder is shown.
- Acceptance Criteria: The UI is visually identical to the current ride request experience when no insights exist.

Test Case 3 — Driver Submits a Pickup Report (Happy Path)

- Action: After completing a ride, the driver fills up the pickup report (3 questions) and selects "No stopping zone" from the predefined tag list.

- Expected: The report is submitted successfully. The driver sees a brief confirmation ("Thanks for the feedback"). The tag is stored and associated with the pickup GPS coordinates.
- Acceptance Criteria: Report submission completes in under 2 seconds. The selected tag is logged in the backend with timestamp, driver ID, and GPS coordinates.

Test Case 4 — Driver Submits Report with Poor Connectivity (Unhappy Path)

- Action: Driver attempts to submit a pickup report while in a low-connectivity area.
- Expected: The app queues the report locally and submits it when connectivity is restored. Driver sees a message: "Report saved - will submit when you're back online."
- Acceptance Criteria: Report is not lost. It is successfully submitted within 5 minutes of connectivity returning. No duplicate submissions occur.

Test Case 5 — Outdated Insight (Edge Case)

- Action: A location had 10+ "Construction delay" reports 6 months ago, but zero reports in the last 60 days.
- Expected: The system deprioritizes or removes the stale insight. No badge is shown for outdated tags.
- Acceptance Criteria: Insights older than 90 days with no new corroborating reports are automatically expired and excluded from the display logic.

Test Case 6 — Insight Badge Tap for Details (Happy Path)

- Action: Driver taps on the insight badge before accepting the ride.
- Expected: A small expandable tooltip or bottom sheet shows the breakdown — e.g., "12 drivers reported: Entrance hard to find (8), No stopping zone (4)."
- Acceptance Criteria: The detail view loads within 1 second. It does not auto-accept or auto-decline the ride. The accept/decline timer continues as normal.

Success Metrics

1. Reduction in pickup-related driver cancellations
 - a. % decrease in cancellations that occur within X minutes of arriving at pickup location
2. Improvement in driver retention (30/90-day)
 - a. Increase in % of drivers still active after onboarding period
3. Engagement with feedback prompts

- a. % of drivers completing post-ride pickup difficulty survey

Compiled Notes of PRD:

Uber Driver App 2026:

<https://www.youtube.com/watch?v=J1rkzeZC5Fc&t=29s>

PRDs

<https://www.productplan.com/glossary/product-requirements-document/>

<https://docs.google.com/document/d/1JmjWa8J9fvSZgdKSFxstJuGVZrO79Vv6tEFwxzaXddo/edit?tab=t.0#heading=h.jgubdj8el47> (example of a PRD)

<https://carlinyuen.medium.com/writing-prds-and-product-requirements-2effdb9c6def> (writing PRD and product requirements)

<https://goberoi.com/on-writing-product-specs-5ca697b992fd>

1. Problem Statement / Business Context

- a. **Problem and its size** - I include **quantitative and/or qualitative data** that I used to size the problem. The goal is to help everyone understand that the problem I'm trying to solve is **worth the investment**.
- b. **Impact** - this section is an extension of the above. But, it focuses more on the impact of solving the problem. I ensure that the impact is always quantifiable and measurable. Even if the math behind the number is not a 100% (and includes assumptions), I still include it. This is my chance to **help engineers appreciate the impact of what we're building**.
- c. **Resources and documents** - I add a lot of links in my PRDs. These links are for resources that I think have extra information that can help readers get more context or details. This works really well in larger teams, where multiple people from multiple teams read your documents.

Eg: An example of a good problem statement looks like this:

"Repeat user orders have dropped by XX% in the last Y months. We see that logins from repeat users have also dropped at the same rate.

Based on the user study <<insert link here>> we did last week, we seem to believe that the current login flow is the main issue."

2. User Persona

Most products have multiple types of users (aka user personas.)

Use this section to specify the user persona(s) that you are focusing on.

I keep this section simple and concise. I include a very basic description of the persona. I link to more detailed documents for those interested.

Key details:

- Personas must be based on user research to accurately represent a product's users. Personas might be made-up people, but they should be based on information about real people.
- Name, age, gender, and a photo
- **Tag line summarizing their character** (Avoid getting too witty, as doing so may taint the persona as being too fun and not a useful tool.)
- **Experience level** with your product or service
- **Context for how they would interact with your product:** Through choice or required by their job? How often would they use it? Do they typically use a desktop computer, their phone, or another device to access it?
- Goals and concerns when they perform relevant tasks: speed, accuracy, thoroughness, or any other **needs that may affect their usage**
- Quotes to sum up the persona's attitude

3. Functional requirements

Meaning: define what a product must do to meet user needs, outlining specific features, behaviors, and user interactions.

- It contains a high degree of details of the feature/product.
- If I am new to the team (and not aware of the team's working style), I try to be as detailed as possible. Extra details can do no harm, but less details will almost always cause damage. I change the level of detail based on my understanding of what the engineers like.
- As an experienced PM, I know that it is critical to **include both the "happy" and "unhappy" flows in the PRD**. Excluding the "unhappy" path - which a lot of young PMs do - is a recipe for disaster. Sample of a happy flow - "User clicks on submit. If the information is correct, redirect the user to the home page." Unhappy flow: "User clicks on submit. If the information is incorrect, show the relevant error message."
- Include user flow diagrams / wireframes

- A good functional requirement usually follows this anatomy:
[ID Number] + [The System/Component] + [Shall/Must] + [Action] + [Condition/Constraint].

Example: FR-02: The system shall redirect the user to the Singpass OIDC provider when the "Login" button is clicked.

Priority Indications

<https://fibery.com/blog/product-management/p0-p1-p2-p3-p4/>

P0 – the site is down and **all work stops, until this issue is resolved.**

- Time is of the essence here.

P1 – finish this task to unblock someone else, required to be done before other things

OR

They're like warning lights on your car's dashboard; ignore them, and you're in for a world of trouble, but you don't have to pull over at this instant.

Eg: Suddenly you're hit with a glitch- user reviews are not displaying correctly on product pages. Now, this isn't a cataclysmic problem; shoppers can still fill their carts and check out. **But hey, reviews are the bread and butter of e-shopping; they guide those critical buying decisions.**

P2 – ordinary flow of work

Imagine you are working on a social media app, and you **plan to introduce a new feature that allows users to filter their feeds based on interests.**

This feature, while a nice addition, **doesn't make or break the app's current usability or satisfaction levels.** It's like adding a chocolate drizzle and cherry on top – great to have, but your ice cream sundae is still good without it.

P2 is about strategic timing and forward-thinking. It's about nurturing these ideas for future growth and user engagement, ensuring they come to fruition at the right moment.

P3 – nice to have, but not required

Think of P3 tasks as those subtle tweaks and enhancements that are more about **refinement than necessity.** They're the kind of improvements that polish your product but don't dramatically change the user experience or functionality.

eg: A P3 task in this context would be the addition of **new aesthetic themes for the app's interface**. Now, while these new themes might make the app look snazzier and give users more personalization options, they aren't directly beefing up the app's core functions, like tracking workouts or crunching health data. So, this kind of task falls into the low-priority category.

P4 – informational only

This level is reserved for tasks that are the **least urgent and have minimal immediate impact** on the project or product. **P4 priorities typically include long-term ideas, low-impact enhancements, or 'wishlist' items.**

These are tasks that would be nice to have but are not essential for the current functionality or success of the product.

Dealing with P4 tasks is about long-term planning and opportunistic development.

4. Non functional requirements (states criteria that evaluate how a system behaves, rather than what it does)

These are obscure requirements that are easy to miss, especially if you are unfamiliar with the product or the user. If we're creating a long flow for a web app, an NFR could be: **"The response time for the backend call that checks the validity of the input should not exceed XXX milliseconds."**

- Latency : Latency is the time delay between a cause and effect, input and output, or stimulus and response in a system
- Security : Create requirements that address data protection, threat resistance, security protocols, and encryption.
- Compliance: Research and address legal or regulatory standards to ensure the system adheres to current and future industry regulations and standards.
 - WCAG (website content accessibility guidelines):
 - POUR
 - Perceivable: Information and user interface components must be presentable to users in ways they can perceive (e.g., **alt text for images, captions for video**).
 - Operable: User interface components and navigation must be operable (e.g., **keyboard accessibility, sufficient time to read**)
 - Understandable: Information and the operation of the user interface must be understandable (e.g., **readable text, predictable navigation**).
 - Robust: Content must be robust enough to be interpreted reliably by a wide variety of user agents, including assistive technologies

- **Eg: aria labels for screen readers**
- Scalability
 - Ability to handle increased workload or data volume
 - [Metric] must be maintained even when [Load/Volume] increases by [Percentage/Amount].
 - Good: "The system shall maintain 99.9% **availability** during peak periods where transaction volume is 3x the **daily average**."
- ACID compliant
 - [Atomicity](#) (All or Nothing): Transactions are treated as a single, indivisible unit; either the entire transaction succeeds, or the whole thing fails and leaves the database unchanged.
 - [Consistency](#) (Validity Rules): A transaction brings the database from one valid state to another, ensuring all data follows predefined rules, constraints, and triggers
 - [Isolation](#) (Concurrent Security): Concurrent transactions do not interfere with each other. The result of running multiple transactions simultaneously is the same as if they were run sequentially.
 - [Durability](#) (Permanent Commitment): Once a transaction is committed, its changes are permanent, even in the event of a power failure or system crash.

5. Testing Plan & Acceptance Criteria

Business test case: "Click on submit button with the incorrect username."

Acceptance criteria: "Clicking on the submit button with the incorrect username should show the relevant error message."

Technical Test cases will be written by engineers

6. Success Metrics

a. Adoption and Reach

- **Adoption Rate:** The percentage of your total user base that has used the new feature at least once.
- **Time to First Action:** How long it takes a user to perform the "key action" (e.g., filing a permit) after logging in for the first time.
- **Feature Discovery:** The percentage of users who clicked on the new feature/module within X days of launch.

b. Efficiency & Performance

- **Task Completion Rate (TCR):** The percentage of users who start a task (like a batch job setup) and successfully finish it without dropping off.
 - **Average Time to Completion:** How long it takes a user to finish a specific workflow.
Success = Decreased time.
 - **Error Rate:** The frequency of "Unhappy Path" triggers (e.g., how often users see a validation error). *Success = Decreased error rate.*
- c. Reliability & System Health (Non-Functional Success)
- **Uptime / Availability:** The percentage of time the system is functional (e.g., "99.9% uptime").
 - **System Latency:** The average response time for a backend call.
 - **Throughput:** The volume of transactions the system handles successfully per hour.
- d. User Satisfaction & Quality
- **CSAT (Customer Satisfaction Score):** Usually a short "How happy are you with this process?" survey after a task is completed.
 - **NPS (Net Promoter Score):** "How likely are you to recommend this tool to a colleague?" (Common in B2B/Trade).
 - **Support Ticket Volume:** The number of helpdesk calls related to the new feature.
Success = Lower volume over time.
7. Assumptions
- What assumptions have been built into EVERYTHING above.